Offer Extraction Pipeline: Solution Overview

Turning scraped AT&T iPhone offer data (JSON) into a structured 29-column CSV, using an LLM plus plain Python rules.

1. The problem I was given

I started with a JSON file holding 12 iPhone products. For each one I had the basics, name, brand, storage variant, price, stock and shipment, plus a list of promotional offers written as loose, unstructured text. Bill credits, switcher rewards, trade-in deals and credit card offers were all mixed in there.

The output had to be a CSV with 29 fixed columns. And every product had to show up twice:

- Row 1: the customer is adding a New Line
- Row 2: the customer is doing an Upgrade

That second part mattered more than it first looked. The same offer can count for one line type and not the other, so I could not just copy a product's offers into both rows. Each row had to be worked out on its own.

2. How I set it up

In short, the pipeline runs in six steps:

Input.json, then load the data and copy the basic fields straight across (no LLM), then gather the offer texts and drop the duplicates, then run the LLM extraction with Gemini (structured JSON output), then check that LLM output, then apply the Python rules for line type, variant and column mapping, then write output.csv, 24 rows by 29 columns.

I kept the LLM's job deliberately small. It only reads and classifies the offer text. Python decides which value goes into which row and column. I did that on purpose: if something looks wrong in the CSV, I can trace it back to a rule I wrote, rather than guessing what a model was thinking.

3. What each step actually does

Copying the basic fields

Product name, brand, variant, MSRP, plan type, stock and shipment come straight from the JSON keys. No model involved, nothing to interpret.

Gathering the offer texts

For each offer I joined the title, description, modal text and card legal text into one block of text, and gave each block a short hash ID. The 12 products hold 48 offers in total, but a lot of them repeat. Only 11 of those texts are unique, so I only needed 11 LLM calls. That saved a fair bit of time and quota.

Letting Gemini read the offers

Each unique text goes to Gemini with a Pydantic schema, so the model has to return valid JSON. I set temperature to 0 to keep the output as steady as possible. For each offer it returns:

- category: device discount, service line, trade-in, switcher or finance
- eligible line types: New Line, Upgrade, or both

- all dollar amounts, copied exactly as written in the text
- the storage variant each amount belongs to, if the offer has a variant table
- whether an amount is monthly
- a short one-line description

The prompt is strict on one point: use only facts from the text, never calculate or round anything, and return null when something is not stated. I also put one example from the assignment brief in the prompt, as a few-shot example.

I saved the results in llm_cache.json. Run the notebook again and it does not call the API for offers it has already done.

Checking the LLM output for made-up numbers

This was the part I did not want to skip. Every amount the LLM returned gets checked against the original text. If an amount does not appear in the text, it gets dropped. Unknown categories get dropped too. In the final run, all the extracted amounts passed this check, which was reassuring, but the check stays in place either way.

Applying the Python rules

For each product and each line type, the rules are:

Line type check. An offer is used only if the current line type is in its eligible list. The $800 switcher offer, for example, shows up only in New Line rows.

Variant check. Device discounts are taken only for the matching storage variant. The 1 TB model gets $628.36, not the 256 GB amount of $660.36. Before comparing, I normalise "256 GB" and "256GB" so a spacing difference does not break the match.

No double counting. Each amount goes into only one column. A monthly amount, for example $5.56/mo., is not used when the total of $200 is available.

Finance offer. For a New Line it is the welcome bonus plus the yearly statement credit plus the bill discount: $200 + $240 + $10 = $450. For an Upgrade it is only the recurring parts: $240 + $10 = $250. I did not add the $20 per billing cycle credit again, because it is already part of the $240 per year.

Money is handled with Python Decimal, not float, so the decimals do not get rounded away by binary floating point.

Writing the output

Before saving, the CSV gets checked: it must have exactly 2 rows per product, and exactly the 29 required columns. Missing values are left blank.

4. When the API pushed back

A few things went wrong while I was building this, and the pipeline had to cope with them:

- Server busy (503), timeouts and malformed JSON: retried up to 3 times, with longer waits each time, 2s, then 4s, then 8s.
- Rate limit per minute (429): wait 60 seconds and retry.
- Daily quota finished (429, per day): retrying will not help there, so the pipeline switches to the next Gemini model in a candidate list.

- If an offer still fails, it is saved as empty, and running the loop again retries only the failed offers.

Both of the first two problems actually happened during development. The API returned 503 several times, and the daily free quota of the first model ran out. The retry and the model fallback handled both, which is why I left them in rather than treating them as one-off fixes.

5. Assumptions I had to make

Some of this was not spelled out cleanly in the brief, so I want to be clear about the calls I made:

- BASE and MSRP both come from pricing.retail, which the brief describes as "base retail" pricing.

- MSRP is written without a "$" sign, as shown in the brief's example. Offer amounts are written with "$".

- Amounts with a comma, such as "1,200", are rewritten as "1200".

- Trade-In Value Type is "Enhanced" for promotional trade-in deals, as returned by the model.

- RESELLER LIST PRICE, PWP/GWP offers, subscription bundle offers and additional offers are left blank, because they are not clearly defined in the brief or they are not present in the data.

6. The result

The final file has 24 rows (12 products times 2 line types) and 29 columns.

I checked it against the iPhone 17 Pro Max 256 GB example from the brief: unique discount $660.36, service line $200, switcher $800, no trade-in value, and finance $450 for a New Line and $250 for an Upgrade. Upgrade rows are blank where the brief expects null.

7. How to run it

1. Open offer_extraction_pipeline.ipynb in Google Colab.

2. Add a Gemini API key in Colab Secrets (the key icon on the left) with the name GEMINI_API_KEY, and turn on notebook access.

3. Upload Input.json and example.json using the Files panel on the left. To skip the API calls, also upload llm_cache.json.

4. Click Runtime, then Run all.

5. The final file, output.csv, appears in the Files panel and can be downloaded.

8. Files in the repository

- offer_extraction_pipeline.ipynb: full pipeline

- Input.json: input data

- example.json: example product from the brief

- llm_cache.json: saved LLM results, so the notebook can run without API calls

- output.csv: final output

- README.md: short summary

- solution_overview.pdf: this document